

Claude Code

Una guía práctica y honesta.

Sin postureo de experto: trucos, referencias sueltas y lo que he aprendido usándolo de verdad.

Track A — fundamentos y criterio · **Track B** — proyecto en vivo

Qué es Claude Code

- Un agente que programa **en tu terminal**, no un chat al lado.
- Lee, escribe, ejecuta, corre tests, usa git — siempre con permiso.
- Disponible como CLI, app de escritorio, web e integración IDE.

Frente a otros agentes de terminal (Hermes, OpenClaw): el ecosistema (skills, hooks, subagentes, MCP) es extrapolable — ahí no está la diferencia. Lo que cambia es el **enfoque**: aquéllos tiran más a **automatizar tareas**, mientras que Claude Code está pensado sobre todo para **programar y trabajar sobre el repo** en profundidad. Y debajo, los **modelos Claude**.

Nota: no es autocompletado. Es un colaborador con acceso a tu máquina.

Instalación — Linux / macOS

Instalador nativo (recomendado):

```
curl -fsSL https://claude.com/install.sh | bash
```

Alternativa, si ya vives en Node:

```
npm install -g @anthropic-ai/claude-code
```

Instalación — Windows

A) Nativo — en PowerShell y listo:

```
irm https://claude.com/install.ps1 | iex
```

B) Con WSL (recomendado) — son dos pasos:

```
wsl --install # 1. en PowerShell: instala Ubuntu  
curl -fsSL https://claude.com/install.sh | bash # 2. ya DENTRO de WSL
```

Ojo: `wsl --install` instala **Linux**, no **Claude Code**.

Acceso: login y plan

- Primer arranque: `claude` → te pide iniciar sesión en el navegador.
- Dos vías de acceso:
 - **Suscripción** (Pro / Max / Team) — cuota incluida, lo normal para el día a día.
 - **API key** — consumo por tokens, para automatizaciones/CI.
- `/login` y `/logout` para cambiar de cuenta.

La cuota: dos ventanas

La suscripción no es barra libre: se consume en dos ventanas.

- **Ventana de 5 h** — arranca con tu primer mensaje y dura 5 horas; al agotarla, esperas al reset.
- **Ventana semanal (7 días)** — límite adicional por encima del de 5 h.
- Ambas se ven en la **statusline**, como `5h:` y `7d:`.

Planificar la sesión (y no compactar/relanzar a lo loco) es también gestionar cuota.

Iniciar y controlar la sesión

```
claude          # arrancar en el proyecto
claude --continue # retomar la última sesión
claude --resume  # elegir una sesión anterior
/init           # generar el CLAUDE.md del proyecto
```

- Puede **instalar dependencias** por ti — pero revisa SIEMPRE contra qué repo/registro va y qué instala.
- **Nunca** lo dejes aceptar sin supervisión aunque se pueda. No es nuestra forma de trabajo: no lo conviertas en una **black-box** que funciona pero no sabes qué hay bajo el capó.

Qué hace `/init` (y qué NO hace)

Hace una sola cosa: recorre el repo y escribe un `CLAUDE.md` en la raíz con lo que ha entendido: estructura, comandos de build/test, convenciones. Si ya existe, propone mejorarlo.

NO crea nada más:

- ni la carpeta `.claude/`
- ni `settings.json` (permisos, hooks, statusline)
- ni agents, skills, commands o hooks

Todo eso lo montas tú — es justo lo que haremos en el proyecto.

Y revísalo: es un **borrador**. Cada línea se paga en cada mensaje.

Dónde vive `.claude`: dos niveles

	Global (usuario)	Proyecto
Ruta	<code>~/ .claude/</code>	<code><proyecto>/ .claude/</code>
Alcance	Todos tus proyectos	Solo ese repo
Se versiona	No	Sí (lo comparte el equipo)
Para qué	Tus preferencias, tu estilo	Reglas del proyecto

Idea clave: **global** = tú, **proyecto** = el equipo.

Qué se crea solo y qué creas tú

Global `~/.claude/` — aparece **solo**, al instalar y arrancar. Pero de fábrica trae poco: tu `settings.json` y `projects/` (historial y memoria por proyecto). Las carpetas `agents/`, `skills/`, `commands/`, `hooks/`, `rules/` **no vienen**: las creas cuando las necesitas.

Proyecto `<repo>/claude/` — **no aparece nunca solo**. Ni `/init` lo crea (recuerda: `/init` solo escribe el `CLAUDE.md`). Lo montas tú.

¿Admite menos que el global? No: **las mismas piezas**. Solo cambian los exclusivos de cada nivel:

- Solo global: `projects/` (historial), `plugins/`.
- Solo proyecto: `settings.local.json` (tuyo, no se versiona).

El árbol global — `~/ .claude/`

```
~/ .claude/  
├── settings.json      # tus ajustes globales  
├── CLAUDE.md          # tu memory personal (aplica a todo)  
├── agents/            # tus subagentes  
├── commands/          # tus comandos /...  
├── skills/            # tus skills  
├── hooks/             # tus scripts de hooks (declarados en settings.json)  
├── rules/             # reglas modulares (*.md), opcionalmente por path  
├── projects/          # historial + memory por proyecto (auto)  
└── plugins/           # extensiones
```

Aquí va lo que quieres en **todos** tus proyectos.

El árbol del proyecto — `<proyecto>/ .claude/`

```
mi-proyecto/  
├── CLAUDE.md          # memory del PROYECTO (en la raíz)  
└── .claude/  
    ├── settings.json  # ajustes compartidos (al git)  
    ├── settings.local.json # tus ajustes locales (NO al git)  
    ├── agents/  
    ├── commands/  
    ├── skills/  
    ├── rules/          # reglas modulares (*.md), opcionalmente por path  
    └── hooks/           # scripts que disparan los hooks
```

Ojo: `settings.local.json` es personal → va a `.gitignore`.

Precedencia: quién gana

De más fuerte a más débil:

1. `managed-settings.json` (lo pone IT; nadie lo sobrescribe)
2. `.claude/settings.local.json` (tu local en el proyecto)
3. `.claude/settings.json` (proyecto, compartido)
4. `~/.claude/settings.json` (global)

Lo del proyecto pisa lo global. Lo local pisa a todo lo tuyo.

Y por encima de todo, la política de la organización.

`settings.json` — el panel de control

Un solo JSON gobierna:

- **permissions** — `allow`: lo que hace sin preguntar. `deny`: lo prohibido siempre. `defaultMode`: con qué modo de permiso **arranca** la sesión (`default`, `acceptEdits`, `plan`, `bypassPermissions`); si no, lo pones a mano con Shift+Tab cada vez.
- **hooks** — scripts que disparan antes (`PreToolUse`) o después (`PostToolUse`) de una herramienta, filtrados por `matcher`.
- **env** — variables de entorno de la sesión (p. ej. telemetría off).
- **statusLine** — la barra de estado. **model** — modelo por defecto.

No es documentación: es lo que el harness **obedece**.

El `settings.json` de nuestro proyecto

```
"permissions": {
  "defaultMode": "default",
  "allow": ["Read(src/**)", "Edit(src/**)", "Read(data/**)",
            "Bash(.venv/bin/python src/*)"],
  "deny":  ["Read(**/.env*)", "Read(**/secrets/**)", "Read(**/*.pem)"]
},
"hooks": {
  "PreToolUse": [{ "matcher": "Write",      "hooks": [proteger-datos] }],
  "PostToolUse": [{ "matcher": "Write|Edit", "hooks": [validar-salida,
                                                         validar-completo, trazar] }]
}
```

- `allow` por patrón: autoriza los scripts, **no** un `python -c "..."` suelto.
- `matcher` decide a qué herramienta se engancha cada hook.

Qué más admite `settings.json`

Nosotros usamos dos claves. Hay muchas más; las que se notan a diario:

- `model`, `effortLevel` — modelo y nivel de esfuerzo por defecto.
- `attribution` — la firma que Claude mete en tus commits y PRs.
- `outputStyle` — cómo responde: rol, tono y formato.
- `autoMemoryEnabled` — la memoria automática, encendida o apagada.
- `cleanupPeriodDays` — cuánto se guarda el historial en disco.
- `agent` — arrancar la sesión **como un subagente** con nombre: hereda su prompt, sus tools y su modelo.
- `editorMode: "vim"` — teclas de vim en el cuadro del prompt.

Añade la clave `$schema` (schemastore) y el editor autocompleta y valida.

Tres que conviene conocer

attribution — por defecto añade `Co-Authored-By: Claude ...` al commit y una línea al PR. String vacío = fuera.

```
"attribution": { "commit": "", "pr": "", "sessionUrl": false }
```

outputStyle — `Explanatory` va explicando el porqué; `Learning` te deja marcadores `TODO(human)` para que escribas tú. Se lee al arrancar: aplica tras `/clear` .

cleanupPeriodDays (30) — no es el login. Es cuánto viven en `~/.claude/` las transcripciones, los planes y los snapshots del rewind.

Y lo que decide IT: `managed-settings.json`

Hay dos formas de que te llegue, y no son la misma cosa.

1. Por login (server-managed) — el Owner la escribe en la consola de `claude.ai`. Al autenticarte con la cuenta de la **organización**, tu CLI se la descarga de `api.anthropic.com` y la cachea. Nada en tu disco, ningún admin. Va atada a la **cuenta**, no al equipo: con tu cuenta personal no te aplica.

2. Por dispositivo (endpoint-managed) — fichero en ruta de sistema o vía MDM: `/etc/claude-code/`, `/Library/Application Support/ClaudeCode/`, `C:\Program Files\ClaudeCode\`. Requiere admin; el SO impide tocarlo.

No se mezclan: si el servidor entrega algo, se ignora lo del dispositivo.

Y lo que decide IT: qué se puede imponer

- `allowedMcpServers` / `deniedMcpServers` — qué MCP se pueden conectar.
- `availableModels` — a qué modelos tiene acceso la gente.
- `claudeMd` — un CLAUDE.md corporativo inyectado en todas las sesiones.
- `allowManagedPermissionRulesOnly` — nadie añade sus propios `allow`.
- `allowManagedHooksOnly`, `disableAllHooks` — control de automatismos.
- `apiKeyHelper` — script que genera la credencial (claves rotatorias).

Se gobierna central: no depende de la buena voluntad de cada uno.

Permisos — reglas de path

```
{  
  "permissions": {  
    "deny": [  
      "Read(**/.env*)",  
      "Read(**/secrets/**)",  
      "Read(**/*.pem)",  
      "Write(**/.env*)"  
    ]  
  }  
}
```

- Son **reglas por path** (glob): `**` = cualquier subdir, `*` = comodín.
- Casan contra la **ruta** del fichero, no contra su contenido.
- Barato, declarativo, sin script. Esto **no** es un hook, es config.

Modos de permiso (y plan mode)

Cada acción se aprueba según el **modo activo**. Cambias con **Shift+Tab**:

- **default** — te pregunta antes de cada acción sensible.
- **plan mode** — Claude investiga y te da un **plan sin tocar nada**; ejecuta solo cuando lo apruebas.
- **accept edits** — acepta ediciones de fichero sin preguntar (tareas de confianza).
- **auto** — un clasificador de seguridad decide las rutinarias.

Plan mode es oro: expone el plan y no toca nada hasta que dices "adelante".
Cero black-box.

Telemetría y tráfico off

```
{  
  "env": {  
    "DISABLE_TELEMETRY": "1",  
    "DISABLE_ERROR_REPORTING": "1",  
    "DISABLE_BUG_COMMAND": "1"  
  }  
}
```

- Granular: apagas telemetría de uso, reporte de errores y `/bug`.
- Todo-en-uno: `CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1` (ojo, también mata el autoupdater).

La barra de estado (statusLine)

Muestra en todo momento:

- Directorio / branch de git
- **Ventana de contexto usada** (clave)
- Modelo activo
- Coste / uso de sesión

Se configura con un script en `statusLine` .

Sirve para saber **cuándo hacer** `/compact` .

/compact — cuándo sí y cuándo no

- **Sí:** contexto casi lleno y vas a seguir la misma tarea.
- **Sí:** cambias de fase y quieres conservar lo esencial.
- **No:** por costumbre o "por si acaso" → pierdes matices.
- **Alternativa:** `/clear` si empiezas algo sin relación.

La barra te dice cuándo. No compactes a ciegas.

CLAUDE.md — instrucciones, no "memoria"

- Instrucciones que **tú escribes** y Claude carga **siempre** en ese ámbito.
Global: tu estilo. Proyecto (`./CLAUDE.md`): convenciones, comandos, "no hagas X".
- Corto y tajante. Nada de novelas: cada línea es contexto que se paga.

Ojo con el nombre:

- No confundir con la **memory** auto (índice `MEMORY.md`): eso son notas que Claude **guarda y recuerda** entre sesiones, no lo que escribes tú.
- Si el CLAUDE.md crece, pártelo en `rules/*.md` ; con frontmatter `paths` cargan solo al tocar ciertos ficheros (lazy, no siempre).

Rules — CLAUDE.md modular y por path

Un `.md` por tema en `rules/`, en vez de un CLAUDE.md gigante:

```
---  
paths: ["src/api/**"]          # opcional: solo carga aquí  
---  
- Las respuestas de la API siempre en snake_case.  
- Nunca romper el contrato de `schemas/`.
```

- Sin `paths` → se carga siempre (como CLAUDE.md).
- Con `paths` → solo cuando Claude toca esos ficheros (lazy = menos ruido).
- Ideal para reglas de un módulo concreto: no molestan al resto.

Memory — lo que Claude recuerda entre sesiones

- Un almacén de **hechos** que Claude guarda y recupera solo: quién eres, decisiones del proyecto, preferencias, referencias.
- Vive en `~/.claude/.../memory/`, con un índice `MEMORY.md`.
- **La escribe Claude**, no tú (aunque puedes pedírselo: "recuerda que...").

Diferencia clave con CLAUDE.md:

	CLAUDE.md / rules	Memory
Quién lo escribe	Tú	Claude
Qué es	Instrucciones	Hechos recordados
Cuándo aplica	Siempre / por path	Cuando es relevante

Las piezas: agent / skill / command / hook / MCP

Pieza	Qué es	Cuándo	Quién dispara
Command	Un prompt guardado <code>/x</code>	Repites una petición	Tú
Skill	Capacidad + ficheros	Proceso reutilizable	LLM (o tú)
Subagente	Contexto propio	Aislar una tarea	LLM o tú
Hook	Script en un evento	Automatizar SIEMPRE	El sistema
MCP	Herramientas externas	Datos fuera del repo	LLM

Los desglosamos uno a uno.

Subagentes

- Sesión con **su propio contexto** (no ensucia el tuyo).
- Ideal para tareas acotadas: clasificar, auditar, buscar.
- Se definen en `agents/*.md` con su rol y herramientas.

Quién lo dispara: las dos vías. Tú lo pides por su nombre, o **lo deduce el LLM** leyendo la `description` del `.md` y viendo que la tarea encaja. Por eso esa descripción es lo más importante del fichero.

Cuándo NO: para todo. Si es una pregunta, pregúntala y ya.

Skills

- Un `SKILL.md` + ficheros de apoyo (plantillas, scripts).
- Claude la **carga sola** cuando la tarea encaja (por su descripción)...
...o la **invocas tú** a mano (`/nombre-skill`).
- Encapsula un proceso repetible: "cómo generamos el informe mensual".

Quién la dispara: por defecto **el LLM**, que lee solo el nombre y la `description` de cada skill y decide si abrirla. Tú puedes forzarla.
Descripción mala = skill que nunca se usa.

Commands

- Un prompt reutilizable en `commands/x.md` → lo invocas tú con `/x`.
- Puede ser una petición simple o un **orquestador**: el prompt le dice a Claude qué pasos dar y en qué orden (lanza agentes, encadena, resume).
- La "lógica" la razona Claude siguiendo el prompt; el command no trae código ni ficheros propios, y no se auto-carga (lo disparas tú).

Ejemplo: `/procesar` orquesta el flujo entero: normaliza → clasificador + auditor en paralelo → informe.

Quién lo dispara: solo tú, escribiendo `/x`. Claude nunca decide lanzar un command. Esa es la diferencia real con la skill.

Hooks

- Un script que se ejecuta en un **evento** (PreToolUse, PostToolUse...).
- Se configuran en `settings.json`, apuntando a `hooks/*.sh`.
- Para cosas que deben pasar **SIEMPRE**, sin que Claude decida.

Quién lo dispara: ni tú ni el LLM. Lo lanza el sistema al ocurrir el evento, y Claude no puede saltárselo ni negociarlo. Por eso es el sitio de lo innegociable.

Criterio (importante): el hook debe hacer algo del **proceso** que el harness NO hace ya. "Validar sintaxis tras editar" es ruido — eso ya lo hace Claude. Un buen hook trata los ficheros o protege el pipeline.

MCP — qué es

- **MCP** (Model Context Protocol) = conectar Claude a **herramientas y datos externos**: una BBDD, una API, el ERP, el gestor documental, el correo.
- Cada servidor MCP expone **tools** que Claude puede invocar.
- Se declaran en `.mcp.json` (proyecto) o en tu config de usuario.

Quién lo dispara: el **LLM**, que elige la tool por su nombre y su docstring. Tú no llamas al MCP: pides el resultado en lenguaje natural. Por eso nombrar bien las tools es diseño, no cosmética.

MCP — consumir uno

Consumir = enchufar uno que **ya existe**: oficial del fabricante, de la comunidad, o hecho por tu equipo. `add` solo lo **registra**, no lo crea.

```
# en la terminal, dentro del proyecto
claude mcp add --scope project <nombre> -- <comando>
/mcp                               # ver estado, autenticar, habilitar/deshabilitar
```

- `--scope`: `local` (solo tú, este proyecto) · `project` (al `.mcp.json`, va al git) · `user` (tú, en todos). El comando solo escribe ese JSON.
- Sus tools aparecen como `mcp__<servidor>__<tool>`.
- Casi siempre pide **credenciales** (token, OAuth): enchufar no es gratis.

Si no existe el que necesitas, toca construirlo → siguiente slide.

MCP — construir uno (desde cero)

Un servidor pequeño que expone tools. Una función decorada **ya es** una tool:

```
from fastmcp import FastMCP
mcp = FastMCP("datos")

@mcp.tool()
def buscar_proveedor(nombre: str) -> dict:
    """Busca un proveedor en la BBDD por nombre."""
    return db.query(nombre)

mcp.run()
```

Nombre + descripción + tipos → Claude sabe **cuándo y cómo** llamarla.

MCP — de dónde sale cada cosa

El protocolo lo implementa el SDK. Tú solo cuidas la **semántica**:

Qué ve Claude	De dónde sale	En el ejemplo
Nombre de la tool	El nombre de la función	<code>buscar_proveedor</code>
Cuándo usarla	El docstring / descripción	"Busca un proveedor..."
Parámetros y tipos	Los type hints → esquema	<code>nombre: str</code>
Nombre del servidor	El que le pones	<code>FastMCP("datos")</code>

- El `mcp__datos__buscar_proveedor` lo compone Claude Code, no tú.
- **No es solo Python:** hay SDK oficial en TypeScript, Go, Rust, Java, C#...

Config vs Hook: el criterio

- ¿Es una regla **estática por path**? → `settings.json` (deny).
- ¿Necesita mirar el **contenido** o el **comando**? → hook.

Ejemplo del error típico:

"voy a hacer un hook para bloquear `.env`" → NO, eso es un `deny`.

"bloquear si el fichero contiene un IBAN real" → SÍ, eso es un hook.

Este matiz separa al que entiende la herramienta del que va a ciegas.

Hype vs necesario

- El modelo **ya infiere** muchísimo. No hace falta micro-instruir.
- Cada regla/skill/hook que añades es **contexto y ruido**.
- Añade una pieza cuando resuelve un dolor **real y repetido**.
- Menos CLAUDE.md tajante > más CLAUDE.md ambiguo.

"¿Esto lo necesito o lo copié de un hilo (Twitter, LinkedIn, Reddit...)?"

Automatización con cabeza: goal / loop

- `/loop` y las tareas tipo "goal" dejan a Claude **repitiendo o persiguiendo un objetivo** solo, en bucle o en background.
- **El peligro:** se dispara el **consumo de tokens**, y peor: actúa sin que mires cada paso → justo la black-box que no queremos.
- **Uso legítimo:** tareas acotadas, verificables y de bajo riesgo (p. ej. "sondea el estado del build cada 5 min" mientras tú miras).
- Regla: automatiza lo que **puedes verificar de un vistazo**. Si no lo supervisas, no lo automatices.

Tips de experiencia

- Un commit por concepto → tu historial es tu documentación.
- Controla el ritmo: "no escribas código todavía, propón primero".
- Deja que proponga estructura antes de picar.
- **Si se desvía, rebobina con `Esc Esc`** en vez de pelearte con el hilo: es tu control de versiones de la conversación.
- Revisa siempre lo que va a borrar/sobrescribir.
- La barra de contexto es tu velocímetro.

Comandos del día a día

Atajo / comando	Para qué
/init	Generar el CLAUDE.md del proyecto
/context	Ver qué ocupa tu ventana de contexto
/compact	Resumir el contexto cuando se llena
/clear	Empezar limpio (tema sin relación)
/mcp	Ver/gestionar servidores MCP
Esc Esc	Rebobinar a un mensaje anterior
Shift+Tab	Cambiar de modo de permiso

Track B — El proyecto

Ahora sí: construimos un sistema real desde un prompt en blanco.

El problema que tenemos

Cada mes llegan documentos **dispersos y en mil formatos**:

- Extractos de banco en **CSV** (cada banco, sus columnas y su encoding).
- Facturas y contratos en **PDF**, con el dato donde le apetece a cada uno.
- Correos y tickets en **texto** suelto.

Y hay que: unificarlos, saber qué es cada gasto, cazar duplicados y cargos raros, y sacar un informe. Hoy: a mano, o con reglas que se rompen en cuanto cambia un formato.

Lo que proponemos

Un solo pipeline que **razona sobre datos sucios**. Cuatro cosas que una herramienta de **reglas fijas** no alcanza:

1. **Formato nuevo en caliente** — sueltas un fichero nunca visto y lo entiende sin que nadie programe un parser.
2. **Clasifica con criterio, no con reglas** — categoría + por qué.
3. **Anomalías narradas** — "3x lo habitual, comercio nuevo", no un flag.
4. **Lo extiendes hablando** — "añade categoría formación" y ocurre en vivo.

Y no queda atado a Claude: lo **construyes con Claude**, pero el pipeline es tuyo (código + prompts). Puedes **ejecutarlo en local** con un modelo abierto — por ejemplo con `opencode` y un modelo más pequeño.

Paso 0 — El prompt semilla

Vamos a construir un procesador de documentos entrantes.

Contexto: recibo un lote heterogéneo de documentos: extractos de banco en CSV, facturas y contratos en PDF, y correos/tickets en texto plano. Quiero normalizarlos a un modelo común, clasificarlos con una categoría y una explicación, detectar duplicados y anomalías, enriquecerlos con datos externos y generar un informe.

Restricciones:

- Python. Simple y legible, sin sobre-ingeniería.
- Datos con PII (IBAN, DNI): no pueden acabar crudos en la salida.

No escribas código todavía. Primero propón la estructura del proyecto y el modelo de datos común.

Paso 0 — Por qué ese prompt

- "No escribas código todavía" → **tú** controlas el ritmo.
- Le doy el *qué* y el *porqué*, no el *cómo*.
- Las restricciones justifican luego configs y hooks.
- El modelo infiere: un parser por fuente, un esquema común...

Menos instrucción, mejor resultado. Eso es el anti-hype en acción.

El proyecto entero, de un vistazo

```
procesador-documentos/
├── CLAUDE.md           # las normas del proyecto
├── .mcp.json           # registro del MCP propio
├── .gitignore          # qué no subimos al repo
├── src/                # el código determinista
├── mcp_gmail/          # nuestro servidor MCP
├── data/entrada/       # el lote a procesar (26 documentos)
│   └── salida/         # lo que produce el pipeline
└── .claude/            # lo que configura a Claude
```

Nada de esto es decorativo. Vamos fichero a fichero.

Los dos directorios que hacen el trabajo

```
src/
├── modelo.py
├── parsear.py
├── normalizar.py
└── informe.py

.claude/
├── settings.json
├── agents/clasificador.md
│   └── auditor.md
├── commands/procesar.md
├── skills/parsear-fuente/
│   └── informe/
└── hooks/ → proteger-datos.sh
            validar-salida.sh + .py
            validar-completo.sh + .py
            trazar.sh
```

A la izquierda, lo **exacto**. A la derecha, el **criterio**. Esa división es la idea central del proyecto. Los dos hooks con `.py` al lado delegan ahí su lógica: el `.sh` engancha el evento, el Python piensa.

Cómo leer las fichas que vienen

Cada fichero, con el mismo esquema:

- **Qué es** — una línea.
- **Necesitamos ...** para que ...
- **Si no:** qué se rompe.
- **Prompt tipo:** cómo se lo pedirías tú, sin picar código.

No hay que memorizar nada: la idea es ver que **cada fichero responde a un problema concreto**, no a una plantilla.

CLAUDE.md · *las normas de la casa*

Qué es: el fichero que Claude lee en cada sesión de este proyecto.

Necesitamos, ya con la estructura acordada, dejarla por escrito para no tener que repetirla en cada sesión nueva.

Prompt tipo — justo después de que proponga la estructura:

"Deja escrito en el CLAUDE.md lo que acabamos de acordar: qué hace el proyecto y el modelo de datos común al que se traduce todo. Añade dos normas: los importes van en Decimal porque son cuentas, y ningún dato personal puede acabar en la carpeta de salida."

Y luego crece: lo que corriges dos veces, baja aquí. La línea del `.venv/bin/python` se añadió el día que un comando falló.

settings.json · *los permisos y los hooks*

Qué es: la configuración que el harness obedece en este proyecto.

Necesitamos que pueda tocar `src/` y leer `data/` sin preguntar, que los secretos estén prohibidos siempre, y declarar los cuatro hooks.

Si no: o pregunta por todo (y los avisos dejan de leerse), o hay barra libre. Ninguna de las dos es aceptable.

Prompt tipo:

"Configura los permisos del proyecto: que pueda editar y ejecutar lo de `src/` sin preguntar cada vez, que leer ficheros de credenciales esté prohibido siempre, y que se ejecuten mis scripts de validación al escribir."

.gitignore · *qué no subimos al repo*

Qué es: la lista de lo que nunca se versiona.

Necesitamos dejar fuera `credentials/` y `token.json` (las llaves del correo), el `.venv/`, y lo que ya genera un script — como `data/salida/`.

Si no: las credenciales del buzón acaban en un repositorio.

Prompt tipo:

"Prepara el `.gitignore` : excluye las credenciales y los tokens del correo, el entorno virtual y la carpeta de salida, que se regenera al procesar."

data/entrada/ · *el banco de pruebas*

Qué es: 26 documentos sintéticos: tres bancos, facturas y contratos en PDF, tickets y correos en texto.

Necesitamos datos **sucios a propósito**: separadores distintos, una codificación antigua, una columna que cambia a mitad, una factura reenviada.

Si no: el pipeline queda demostrado con datos perfectos, que no existen.

En un flujo real no es una carpeta: los documentos llegan de sitios distintos — carpetas de red, un gestor documental, el correo. Cualquiera de esas fuentes se engancha con un **MCP**, como haremos con Gmail.

`src/modelo.py` · *la ficha única*

Qué es: el `Registro` al que se traduce todo, con su `id` calculado por contenido (fecha + importe + contraparte).

Necesitamos un solo idioma interno, y una huella que **una el mismo movimiento** aunque llegue por dos sitios.

Si no: cada fase inventa su formato, y el mismo pago en dos bancos cuenta como dos gastos distintos.

Prompt tipo:

"Define la ficha común a la que se traduce cualquier documento, con un identificador calculado a partir del contenido, no del fichero, para que un mismo movimiento que aparezca en dos sitios se reconozca como el mismo."

skills/parsear-fuente/ · *la regla escrita una vez*

Qué es: una skill que Claude carga solo cuando toca analizar una fuente.

Necesitamos fijar de una vez las reglas: probar codificaciones, detectar el separador, gasto en negativo, y que una fila rota no tire el fichero.

Si no: hay que repetir las instrucciones y cada analizador sale distinto.

Prompt tipo:

"Fija como norma reutilizable el procedimiento para normalizar una fuente nueva, de modo que se aplique automáticamente cada vez que se añada un analizador."

`src/parsear.py` · *el traductor*

Qué es: un parser por familia de fuente (CSV/TSV, PDF, texto) y un despachador que elige según la extensión.

Necesitamos leer datos reales: BBVA en codificación antigua, La Caixa con una columna nueva a mitad, cada banco llamando distinto a la misma columna.

Si no: el primer fichero raro rompe la ejecución entera.

Prompt tipo:

"Escribe los analizadores siguiendo esa norma. Los ficheros vienen sucios, con separadores y codificaciones distintas, y alguno cambia de estructura a mitad. Que se salte lo que no pueda leer, pero que deje registrado qué se ha saltado y por qué."

"¿Y si se salta un documento?"

Se salta, sí. En silencio, **no**: cada descarte deja fichero, línea y motivo en `descartes.json`, y el informe abre con el recuento.

```
- Filas no leídas: 1 (lacaixa_export.tsv) – detalle en descartes.json  
  → línea 14: "cabecera repetida a mitad del fichero"
```

¿Quién decide el motivo? El **código**, no el modelo: son cuatro casos fijos (cabecera repetida, faltan columnas, fila incompleta, extensión no soportada). Determinista y comparable entre meses.

Saltar para no morir en el primer fichero raro; registrar para no perder nada.

`src/normalizar.py` · *la fase de ingesta*

Qué es: recorre `data/entrada/`, parsea todo y vuelca `data/salida/normalizado.json`.

Necesitamos un punto de corte claro: a partir de aquí, **nadie vuelve a tocar los documentos crudos**. Las fases siguientes leen ese JSON.

Si no: cada agente abre los PDF por su cuenta, gasta contexto y cada uno entiende una cosa.

Prompt tipo:

"Implementa la fase de ingesta: recorre la carpeta de entrada, aplica los analizadores y consolida el resultado en un único fichero, que será la entrada de las fases siguientes."

`agents/clasificador.md` · *criterio, no lista*

Qué es: un subagente con contexto propio y solo `Read` y `Write`, que categoriza cada ficha y explica por qué.

Necesitamos que **razone por el objeto del gasto**, no por listas de proveedores, y con un desempate fijo.

Si no: o se mantiene una lista infinita de comercios, o la misma factura cambia de categoría según el día.

Prompt tipo:

"Asigna a cada ficha una categoría de gasto con su justificación. El criterio es el objeto del gasto, no el sector de la contraparte, y el desempate debe ser fijo: dos ejecuciones sobre los mismos datos dan el mismo reparto."

agents/auditor.md · *el que sospecha*

Qué es: un segundo subagente que, en paralelo, escribe las anomalías.

Necesitamos que cace lo que un hash no ve: la factura **reenviada** con otra fecha, el cargo tres veces mayor de lo normal, el proveedor nuevo con un importe alto.

Si no: solo se detectan duplicados exactos, los menos peligrosos.

Prompt tipo:

"En paralelo, una segunda revisión independiente que señale duplicados y movimientos anómalos, cada uno explicado en una frase. Debe cubrir también los duplicados no idénticos, como una factura reenviada con otra fecha."

Los agentes solo devuelven lo que aportan

Ni fecha, ni importe, ni contraparte: eso ya está en `normalizado.json`. Cada agente escribe su parte, y el informe las vuelve a juntar por el `id`.

```
clasificador → { id, categoria, explicacion }  
auditor      → { id, anomalias }
```

Por qué: al reemitirla completa, se pasaban del límite de salida y se perdían registros por el camino.

Cómo se arregló: cambiando el **contrato**, no dando más permisos ni subiendo límites. El informe ya cruzaba por `id`.

hooks/proteger-datos.sh · *PreToolUse*

Qué es: antes de escribir en `data/salida/`, mira el contenido y frena si hay un IBAN o un DNI en claro.

Necesitamos mirar **lo que se va a escribir**, no la ruta. Un `deny` por path no puede: la carpeta es legítima, lo que no lo es va dentro.

Caso real — `data/entrada/ticket_gasto_11.txt` trae la cuenta y el DNI **en el asunto**, y el asunto acaba en la descripción de la ficha:

```
Asunto: Reembolso dietas ES9021000418401234567891 - DNI 45872913K
```

Si el agente copia esa descripción a su salida, el hook **corta la escritura** y le pide anonimizarla. Se ve pasar en vivo.

validar-salida.sh + su **.py** · *PostToolUse*

Qué es: dos ficheros para un hook: el **.sh** filtra y decide, el **.py** valida el esquema de cada artefacto.

Necesitamos que cada JSON cumpla **su** esquema: la ficha completa en **normalizado.json**, y solo su parte en los otros dos.

Si no: un JSON roto pasa a la fase siguiente y el fallo aparece tres pasos después, donde ya no se entiende.

Por qué dos ficheros — hacen cosas distintas: el **.sh** es el **enganche** con Claude Code (es lo que se declara en **settings.json**); el **.py** es quien **revisa el contenido**. Bash filtra rutas bien; validar esquemas, no.

Cómo se pasan el testigo el `.sh` y el `.py`

```
Claude escribe clasificado.json
  ↓ Claude Code lanza el hook y le pasa el evento en JSON por la entrada
validar-salida.sh      ¿es un .json de data/salida/?  no → exit 0 (calla)
  ↓ sí: llama al Python con la ruta del fichero
_validar_salida.py     abre el fichero y valida el esquema
  ↓ ¿mal? imprime el error y termina con código 1
validar-salida.sh      recoge ese error y hace exit 2
  ↓
Claude LO LEE y reescribe el fichero corregido
```

El `.sh` decide **si toca actuar** y traduce el resultado a lo único que Claude Code entiende: el **código de salida**. `exit 2` es "corrígelo".

`validar-completo.sh` + su `.py` · *cobertura*

Qué es: compara los `id` de `normalizado.json` con los de clasificado/auditado, y para si falta alguno.

Necesitamos cerrar el fallo **silencioso**: que un agente deje registros sin procesar y nadie se entere.

Si no: el informe sale, cuadra a la vista, y le faltan fichas. El peor error posible: el que no avisa.

Cómo actúa: devuelve la lista de `id` que faltan y el agente completa solo lo que dejó suelto. Compara identificadores, no cantidades.

`hooks/trazar.sh` · *la bitácora*

Qué es: apunta en `data/salida/_proceso.log` qué artefacto se escribió y cuándo.

Necesitamos poder responder "¿esto de cuándo es?" sin adivinar.

Si no: quedan cuatro JSON sin saber si son de la misma pasada.

Prompt tipo:

"Registra en un log cada fichero de salida que se genere, con su nombre y la hora."

Es el hook más simple del proyecto, y el que más veces salva una discusión.

skills/informe/ · *el entregable definido*

Qué es: la skill que fija qué debe contener el informe y con qué reglas.

Necesitamos dejar escrito el **formato del entregable**: resumen, tabla por categoría, anomalías con su origen, sin jerga y sin PII.

Si no: cada informe sale distinto y no se pueden comparar dos meses.

Prompt tipo:

"Define el formato del informe mensual: resumen con totales, desglose por categoría y lista de avisos con su documento de origen. En lenguaje de negocio y sin ningún dato personal."

`src/informe.py` · *los números, en código*

Qué es: cruza los tres JSON por `id`, agrega y escribe `informe.md`.
Junta, no recalcula.

Necesitamos que las sumas las haga **código**: los totales tienen que dar lo mismo en cada pasada.

Si no: es el modelo quien suma los importes. Aunque acierte, no hay forma de demostrar que acertó.

Detalle real: si falta una fase, el informe **se genera igual** y avisa de lo que falta. Degradar bien también se diseña.

`mcp_gmail/server.py` · *la capacidad que no venía*

Qué es: un servidor MCP con la tool `traer_facturas(asunto)` : busca los correos, baja el **adjunto binario** y lo deja en `data/entrada/`.

Necesitamos el fichero tal cual (`%PDF` , sus bytes). El conector de fábrica apunta a tu cuenta personal, y solo lee y busca: no lo descarga.

Si no: alguien baja los adjuntos a mano cada mes.

Prompt tipo:

"Conecta con el buzón donde llegan las facturas y descarga a la carpeta de entrada los adjuntos de los correos que coincidan con un asunto. Permiso de lectura únicamente: no debe poder enviar."

Montarlo entero: `README.md` + `credentials/`

Es la pieza con más pasos, y son los mismos para cualquier MCP con OAuth:

1. **Servidor** — `mcp_gmail/server.py` con la tool `traer_facturas`.
2. **Credenciales** — en Google Cloud: habilitar la API → pantalla de consentimiento → cliente OAuth de escritorio → JSON a `credentials/`.
3. **Primer login** — arrancar una vez, autorizar en el navegador, y se guarda el `token.json`.
4. **Registro** — la entrada en `.mcp.json`; `/mcp` lo muestra conectado.
5. **Prueba** — `traer_facturas("FACTURA DEMO")` → PDF en `data/entrada/`.

El detalle, en `mcp_gmail/README.md`. Y `credentials/` queda fuera del repo por **dos** vías: el `.gitignore` y el `deny`.

Diseñarlo tú: lo que ganas

- **El binario puro** — te llevas el PDF tal cual (`%PDF` , streams), no una interpretación. Es lo que tu parser necesita para procesarlo de verdad.
- **Solo las tools que quieras** — reenviar, responder, subir a Drive/S3, avisar por Slack... cada capacidad = una función más. Se extiende hablando.
- **Tú fijas los permisos** — hoy `gmail.readonly` (solo lee). Enviar exige añadir el scope a propósito: concedes lo justo, no todo por defecto.
- **Controlas la superficie** — sabes exactamente qué hace y a qué llega.

Contra caja: leer es de bajo riesgo; **enviar/exportar fuera** ya tiene alcance real → ahí entran los **hooks de seguridad**. Poder ≠ dejarlo suelto.

`.mcp.json` · *el enchufe*

Qué es: ocho líneas que le dicen a Claude Code cómo arrancar el servidor. Va en la **raíz** del proyecto: nombre y sitio fijos, ahí lo busca.

```
{ "mcpServers": {  
  "gmail-facturas": {  
    "command": ".venv/bin/python",  
    "args": ["mcp_gmail/server.py"] } } }
```

Es solo el **registro**, no el código: un único fichero para todos tus MCP. Cada servidor vive en su carpeta (`mcp_gmail/`, y mañana los que sumes).

commands/procesar.md · *el guion*

Qué es: el prompt guardado que orquesta todo: normaliza → los dos agentes en paralelo → informe → resumen.

Necesitamos que el orden esté escrito y no dependa de cómo lo pidas ese día, incluyendo el "lanza los dos a la vez, no esperes".

Si no: cada ejecución es distinta y el paralelismo se pierde.

Prompt tipo:

"Deja guardado un comando que ejecute el proceso completo: normalizar, lanzar las dos revisiones en paralelo, generar el informe y cerrar con un resumen. Si una fase falla, que se detenga y lo indique."

`data/salida/` · *lo que queda al final*

Cuatro artefactos y un log, cada uno con su dueño:

- `normalizado.json` — todas las fichas · lo escribe **el código**.
- `descartes.json` — lo que no se pudo leer, con motivo · **el código**.
- `clasificado.json` — `{id, categoria, explicacion}` · **clasificador**.
- `auditado.json` — `{id, anomalias}` · **auditor**.
- `informe.md` — el entregable · **el código**, cruzando por `id` los tres primeros.
- `_proceso.log` — qué se generó y cuándo · **el hook**.

Se puede borrar la carpeta entera y reconstruirla con un comando. Nada de esto es un original que haya que custodiar.

Ver el consumo (anti black-box)

Tras cada pieza, un gesto de 5 segundos:

- `/context` → cuánto ocupa cada skill / CLAUDE.md / MCP (los MCP van *deferred* = casi 0).
- **statusline** `ctx: x%` → antes/después de un subagente: tu ventana no sube, gasta la suya.
- `/mcp` → servidores y sus tools.

Ves el coste, decides con criterio. No es una caja negra.

Construcción en vivo — la hoja de ruta

Desde un prompt en blanco, cada pieza = **un commit**:

1. **Las normas** — `settings.json` + `CLAUDE.md`, con los datos ya puestos.
2. **La ficha común** — skill `parsear-fuente`, modelo y parsers → todo el lote hablando el mismo idioma.
3. **El criterio** — `clasificador` y `auditor`, lanzados **en paralelo**; mira que tu `ctx:` no sube.
4. **Lo innegociable** — los cuatro hooks.
5. **El entregable** — skill `informe` y su código.
6. **El disparador** — `/procesar`: un tiro encadena todo.
7. **El MCP propio** — servidor, credenciales y registro.

Leer el informe: de dónde sale cada cosa

Cada dato del `informe.md` tiene un dueño:

En el informe	De dónde sale
Documentos y reparto por fuente	<code>normalizar.py</code> · <i>código</i>
Filas no leídas y su motivo	<code>descartes.json</code> · <i>código</i>
Totales de gasto / ingreso / saldo	<code>informe.py</code> · <i>código</i>
<code>id</code> de cada ficha	hash de contenido · <i>código</i>
Categoría + explicación	subagente clasificador · <i>criterio</i>
Anomalías y duplicados	subagente auditor · <i>criterio</i>

Los **números** son código (exactos); el **criterio** lo ponen los agentes.

Cierre

- La herramienta no sustituye pensar; hace que **pensar rinda** mucho más. El criterio sigue siendo tuyo: qué se automatiza y qué no, es una decisión.
- **Supervisar no es opcional.** Ves lo que hace y lo que cuesta, y por eso puedes corregirlo. El día que dejes de mirar, deja de ser tu trabajo.
- **Empieza por un dolor real**, no por montar piezas. Cada pieza que añades se paga; si no resuelve algo que duele hoy, sobra.
- El repo queda **disponible para clonar**: es más fácil entenderlo tocándolo que viéndolo en una pantalla.